

FORGE

A Certificate-First Design

Beyond Lean's grind

Demand-directed proof search, nonlinear certificates, induction, witness synthesis, and checked solver cooperation

Research design and executable prototype report

14 September 2026 / Reference target: Lean 4.34.0

Abstract

A tactic substantially broader than `grind` should not replace its congruence closure with another e-graph, nor merely run a longer list of existing tactics. It should retain `grind` as a strong local engine while discovering the missing intermediate facts, structural arguments, witnesses, and nonlinear certificates that make local reasoning succeed. This report specifies *Forge*, a typed, scope-aware, certificate-first architecture for that purpose. Four research prototypes are implemented and tested: demand-directed Horn inference, sparse polynomial-cone search, Bernstein positivity certificates, and polynomial recurrence invariants. They demonstrate concrete mechanisms, including their failure modes, but do not establish superiority over the actual Lean tactic. The proposed Lean integration has not been implemented or benchmarked.

Evidence boundary. The recorded run has 73 passing Python test cases, 109 accepted nonlinear certificates, 43 successful Bernstein certificates out of 45 cases, and 49 recurrence invariants. Generated Lean replay source is included, but was **not compiled or kernel-checked**: no Lean/Lake runtime was available. This package is a detailed design plus research prototypes, not a released implementation of `forge`.

Prepared for the requested investigation of stronger Lean automation. Source locations, exact dependency versions, raw measurements, tests, and reproduction scripts accompany the article.

Contents

1	The design decision	4
1.1	What should become easier	4
1.2	Implemented, specified, and still to be built	5
2	Begin with the actual baseline	5
2.1	Do not underestimate current <code>grind</code>	5
2.2	Related systems to reuse rather than relabel	5
2.3	Version and provenance	6
3	Architecture: proof obligations rather than opaque tactic calls	6
3.1	The persistent state	6
3.2	Scope, identity, and transport	7
3.3	The cooperative action protocol	7
3.4	A precise fallback guarantee	8
4	Demand-directed instantiation and intermediate lemmas	8
4.1	Backward demands with forward proof completion	8
4.2	Tabling prevents cyclic pseudo-proofs	8
4.3	A family that isolates the advantage	9
4.4	Library retrieval and induction of useful demands	9
5	PolyCone: search for nonlinear inequality certificates	10
5.1	Certificate language and its soundness	10
5.2	Finite dictionary construction	10
5.3	Search becomes linear algebra with an untrusted oracle	11
5.4	Worked certificates	12
5.5	Theory cooperation through generated inequalities	12
5.6	Limits and a concrete SOS escalation	13
6	Bernstein certificates on rational boxes	13
6.1	A complementary proof language	13
6.2	Subdivision is a proof tree, not sampling	14
6.3	Why true statements can remain unresolved	14
7	Induction as a search action	14
7.1	Generate a justified induction problem	14
7.2	Worked accumulator example	15
7.3	Failure-guided auxiliary lemmas	16
8	Polynomial invariant synthesis with exact validation	16
8.1	From recurrence data to a theorem candidate	16
8.2	Example: sum of squares	16
8.3	Generalizing the invariant engine	17
9	Existential witnesses and counterexample-guided synthesis	18
9.1	Construct witnesses in the correct scope	18
9.2	A concrete CEGIS loop	18
9.3	Models guide search; they do not certify theorems	19
10	Certified solver islands and theory combination	19
10.1	Export a fragment, not an untyped approximation of Lean	19
10.2	Do not import unsound combination assumptions	19
10.3	Two trust profiles	19
11	Concrete Lean integration plan	20

11.1 Use the current extension interface	20
11.2 Module boundaries and required contracts	21
11.3 Libraries and algorithms with specific roles	21
11.4 Resource allocation and proof-cost awareness	22
11.5 User-facing syntax and explanations	22
12 Soundness obligations for the whole system	23
13 Experiments actually performed	23
13.1 Environment and reproducibility	23
13.2 Summary of observed outcomes	24
13.3 Demand search: counts before speed claims	24
13.4 Nonlinear cases, including unresolved truths	25
13.5 Bernstein and invariant evidence	25
13.6 What the Lean export does and does not establish	25
14 A fair evaluation for a real Forge implementation	26
14.1 Benchmark strata and leak prevention	26
14.2 Baselines and ablations	26
15 Development sequence and acceptance gates	26
16 Conclusion	27
A Certificate formats and independent checking	27
A.1 Sparse polynomials	27
A.2 Cone and invariant records	28
A.3 Horn and box records	28
B Reproducing the package	29
References and source provenance	30

1 The design decision

Build a proof coordinator around *grind*, not a replacement for its successful core.

The proposed tactic, *forge*, uses a protected stock-search lane and a richer search lane. The latter combines backward demands, checked forward consequences, bounded structural proof search, and specialized certificate producers. Crucially, those producers can return *useful lemmas*, not only a verdict on the entire goal. An inequality certificate can supply an induction step; an induction lemma can expose arithmetic facts; a synthesized witness can turn a quantified problem into a local *grind* call.

“More powerful” has three distinct meanings. *Coverage* concerns which proof patterns the tactic can discover. *Efficiency* concerns resources used on common goals. *Reliability* concerns successful replay, predictable failure, and the trusted assumptions of the resulting theorem. Forge targets all three, but this report provides experimental evidence only for isolated Python mechanisms. It does not claim measured Lean-level coverage gains or a universal speedup.

1.1 What should become easier

The intended gains are concentrated where an otherwise short proof needs one missing idea: strengthening an induction hypothesis, manufacturing a useful term absent from the ground context, choosing a product of known nonnegative expressions, finding an existential witness, or extracting a finite Boolean subproblem. A large library can contain all the necessary lemmas and still leave these decisions unresolved.

For example, from $x \geq 1$, $y \geq 2$, and $z \geq 3$, a useful intermediate fact is

$$(x - 1)(y - 2)(z - 3) \geq 0.$$

This is not difficult mathematics. The automation problem is selecting that product, recording its dependencies, and making it available when a polynomial normalizer or another solver needs it. Likewise, a proof about an accumulator function may need a universally quantified accumulator invariant before any amount of ground simplification helps.

The proposed architecture makes those choices explicit, bounded, inspectable, and replayable. It does not promise automatic discovery of arbitrary creative mathematics, complete higher-order proof search under finite resources, or complete nonlinear arithmetic from a fixed-degree certificate basis.

1.2 Implemented, specified, and still to be built

Component	Delivered evidence	Production work still needed
Demand-directed inference	Python tabling, dependency closure, proof DAG checker, differential tests	Typed Lean terms, e-class matching, library index, scoped metavariables
Nonlinear cone	Exact sparse identities; LP proposals; rational reconstruction	Lean reification, checker theorem or proof replay integration, scheduling
Bernstein positivity	Exact box certificates and complete-cover checking	Lean basis lemmas, dependent proof replay, richer split policies
Recurrence invariants	Interpolation proposals; exact base and transition identities	Extract Lean recursive definitions and apply appropriate recursors
Induction and witnesses	Detailed algorithms and worked derivations	Implement search, generalization, typed candidate synthesis
SAT/SMT/superposition	Concrete adapters and trust contracts specified	Version-compatible integrations, proof-rule coverage, testing
Whole <code>forge</code> tactic	Architecture, interface, acceptance protocol	Not implemented; no Lean head-to-head results

2 Begin with the actual baseline

2.1 Do not underestimate current grind

The current reference describes a cooperating collection of congruence closure, constraint propagation, E-matching, guided case analysis, and theory solvers. Its examples include nontrivial polynomial identities and finite-field reasoning, not merely linear inequalities. It builds ordinary Lean proof terms. The documentation also identifies large combinatorial Boolean search as a poor fit and recommends a SAT-backed route for such problems [1].

Consequently, “add nonlinear arithmetic” is too imprecise as a contribution. Forge’s algebraic extension is *search for nonnegativity certificates and useful inequality cuts*, rather than rediscovering polynomial equality normalization. Similarly, “add an e-graph,” “share facts,” and “support plugins” describe capabilities already present, not substantive extensions.

The inspected implementation and API also contain theorem guards, delayed instances, support for extensionality and function congruence, model-based theory-combination hooks, and a solver-extension interface. The Action control layer uses continuations to accommodate branching, nonchronological backtracking, and replay-script extraction [2, 3]. Forge should exploit those mechanisms instead of replacing them with an incompatible second notion of proof state.

2.2 Related systems to reuse rather than relabel

Aesop supplies a useful model for normalization rules and safe/unsafe AND-OR search [4]. Mathlib’s `linarith` already separates an untrusted search oracle from certificate reconstruction; `nlinarith` adds nonlinear preprocessing [5]. Thus, an LP oracle followed by an exact identity

check is not itself novel. The extra proposal is a goal-sensitive product-and-square dictionary, cooperation with structural search, and the protocol for feeding certified consequences back into a persistent context.

Duper is a concrete candidate for bounded superposition-based reasoning, including higher-order capabilities [6]. Lean-SMT demonstrates a route through preprocessing and proof-producing external SMT search, with Lean-side reconstruction [7]. These systems are optional lanes, not reasons to erase type information or trust an external solver’s answer. A dependency must pass a compatibility and proof-reconstruction test before it is enabled; the mere existence of a repository does not establish compatibility with the target toolchain.

Recent work on interventions inside `grind` is particularly relevant. It reports that always-on changes can rescue some problems while breaking other previously successful searches, and advocates interventions after baseline failure [8]. Forge adopts that deployment discipline. It does not claim that a fallback cascade or bounded lookahead is a new invention.

2.3 Version and provenance

The inspected Lean release is **4.34.0**. The supplied replay project pins `mathlib` to commit

1cf325a0cf67aca2b04d76b5380ff6a9e410aeffa.

The upstream toolchain at that revision specifies Lean 4.34.0. These facts were read on 14 September 2026 in the America/Los_Angeles time zone [9, 10]. The corresponding commit timestamp can fall on 15 September in UTC. Pinning a compatible-looking pair is not a successful build: the generated project was not compiled here.

3 Architecture: proof obligations rather than opaque tactic calls

3.1 The persistent state

At a logical level a search node is a sequent $\Gamma \vdash G$, together with a resource account and a persistent snapshot. Its auxiliary data are:

- a typed term index and the local `grind` state, with equality explanations;
- certified facts, pending guarded facts, and backward demands;
- an AND–OR graph of proof obligations and alternatives, with dependency and scope information;
- a certificate store, learned failure summaries, and a replay-cost estimate.

The distinction between *fact* and *demand* is fundamental. A fact contains an expression p and a proof of p in a specified context. A demand merely asks for such a proof. A proposed lemma is not inserted into the fact store until it has been proved. Otherwise, two unsuccessful searches could “prove” each other through circular assumptions.

Represent a guarded proposal as

$$(\Gamma; g_1, \dots, g_k; p; c), \quad c : \Gamma \vdash g_1 \rightarrow \dots \rightarrow g_k \rightarrow p.$$

The watchers for the g_i may awaken when equalities or arithmetic bounds change. Once each guard has a proof in a permitted scope, ordinary application of c produces a fact. Before that

point, p is only a conditional proposal. This is also how a division normalization can wait for a nonzero denominator instead of silently assuming it.

3.2 Scope, identity, and transport

Every stored theorem application records its local-context identifier, free-variable dependencies, instantiated universes, typeclass instances, metavariable assignments, and proof parents. A fact valid under a branch assumption cannot leak into its sibling. To move a fact outside its original scope, first abstract the branch assumptions and then prove the resulting implication in the receiving context.

For example, a branch may prove $x^{-1}x = 1$ using $x \neq 0$. After backtracking, the reusable result is $x \neq 0 \rightarrow x^{-1}x = 1$, not the unguarded equality. Caching only the printed expression loses this distinction. Environment revision, transparency policy, and dependency closure must be part of a cache key or explicitly revalidated on lookup.

Lean expressions also cannot be treated as an untyped term algebra. Homogeneous equality requires compatible types; dependent applications may need heterogeneous equality and transport. Existing `grind` services should supply equality explanations. Forge should never identify terms just because their pretty-printed strings agree, or because an external first-order encoding mapped them to the same symbol.

3.3 The cooperative action protocol

A plugin returns one of four logical outcomes: no applicable action, checked progress, a closed proof, or an unresolved result with a reason. A progress result carries a certified lemma or a well-formed refinement of the goal into subgoals. Failure, numerical trouble, unsupported syntax, and timeout do not mean falsehood.

```
cooperate(node, budget):
  saturate cheap certified consequences
  if the goal has a checked proof: return that proof
  enqueue demands extracted from the goal and pending guards
  repeat while resources remain:
    choose an action with a bounded local budget
    run it against an isolated snapshot
    if it returns a certificate:
      reconstruct and check its exact proposition
      merge only scoped, certified consequences
    if it returns subgoals:
      register an AND-node and its proof-combining continuation
    if it closes an OR-alternative:
      retain its proof and cancel unnecessary siblings
  age postponed actions; periodically service the fair lane
  return unknown with residual obligations and resource statistics
```

This is stronger than repeatedly restarting `first | grind | nlinarith | aesop`: the plugins exchange selected lemmas, guard proofs, and demand information. Nevertheless, the first usable integration can begin with independent leaf calls while the shared-state adapter is tested. That engineering milestone should not be advertised as the complete architecture.

3.4 A precise fallback guarantee

Let $B(G, H)$ denote the result of stock `grind` on G under a fixed budget H and a fixed environment. Run that exact computation first. If it succeeds, return its proof. Only if it fails, restore the initial snapshot and spend an additional budget K on Forge’s extensions.

Proposition 3.1 (Additive-budget preservation). *Assuming identical baseline inputs, resource accounting, and execution conditions, the cascade succeeds on every goal for which $B(G, H)$ succeeds.*

Proof. The successful baseline execution is performed unchanged, and its result is returned before any extension is run. $\square$

This elementary guarantee is about *additional resources*. It is not a proof that allocating part of the same total budget H to new machinery preserves all baseline successes. Nor does a simultaneously running baseline thread receive an identical wall-clock or memory budget if the other threads contend with it. Report additive-budget and fixed-total-budget experiments separately. Fairness prevents starvation in an unbounded search; it does not make a bad finite-budget allocation free.

4 Demand-directed instantiation and intermediate lemmas

4.1 Backward demands with forward proof completion

Suppose a theorem has an instantiated conclusion matching the wanted fact:

$$A_1(\vec{t}) \rightarrow \cdots \rightarrow A_k(\vec{t}) \rightarrow C(\vec{t}).$$

Rather than forward-generating every instance of every theorem, create demands for the $A_i(\vec{t})$ when $C(\vec{t})$ is useful. A conclusion index retrieves candidate declarations by typed head symbols, argument structure, and relevant relation classes. Matching is modulo certified equality when useful, but elaboration decides whether the proposed instantiation is legal.

A candidate whose conclusion does not determine every parameter needs a second phase: enumerate a bounded typed term grammar, solve a small witness problem, or leave it to the fair forward lane. It must not arbitrarily instantiate a universally quantified theorem variable with an ill-scoped metavariable.

The Python prototype deliberately avoids that difficulty. Its rules are single-sort Horn clauses whose body variables all occur in the head. Matching a *ground* demanded head therefore determines the substitution for every premise. There is no higher-order unification, dependent typing, equality reasoning, or library retrieval in this prototype.

4.2 Tabling prevents cyclic pseudo-proofs

Build a finite, budgeted backward closure of ground demands and candidate rule instances. Then propagate actual proofs forward over that dependency graph. Each rule instance has a counter of unresolved premises. Input facts initialize the agenda; proving a premise decreases the counters of its dependents. A rule fires only when every premise is proved. Store the proof node at that moment.

This is not a depth-first convention that treats “currently being visited” as success. Rules $P \rightarrow Q$ and $Q \rightarrow P$, without a base fact, prove neither. The certificate is a topologically ordered DAG: every premise index is strictly smaller than its conclusion’s index. Its checker reconstructs the exact substitution and conclusion for each theorem application.

Proposition 4.1 (Relative completeness of the finite Horn core). *For a fixed finite set of ground candidate rule instances, if the dependency agenda is exhausted without a resource cutoff, every fact in their least forward closure is eventually proved. In particular, a backward-relevant closure containing a ground derivation of the target will discover a proof.*

Proof. Induct on the height of a finite derivation. Height-zero input facts enter the agenda initially. Once all smaller-height premises of an instance have entered, its unresolved counter reaches zero, so its conclusion is enqueued. Tabling ensures repeated discoveries do not obstruct this propagation. $\square$

The statement concerns the fixed finite closure, not unrestricted first-order logic. A depth cap can exclude a necessary derivation. With function symbols, the full relevant closure may be infinite. Such outcomes are reported as `unknown`, not as refutations.

4.3 A family that isolates the advantage

Take a base fact $P(a)$ and the two rules

$$P(x) \rightarrow P(g(x)), \quad P(x) \rightarrow P(f(x)).$$

Ask for $P(f^d(a))$. In the supplied breadth-first forward implementation, the g -rule precedes the f -rule. The goal is therefore the last word generated at depth d , after

$$1 + 2 + \cdots + 2^d = 2^{d+1} - 1$$

facts. Backward demand generation produces exactly the $d + 1$ facts on the f -chain. Both returned proofs contain $d + 1$ nodes.

At depth 14 the measured counts are 32,767 versus 15. This is an intentionally adversarial *Python Horn experiment*, not a reconstruction of `grind`. Current `grind` already sees a negated goal and performs sophisticated instantiation; this example cannot establish that it would suffer the same expansion. What the experiment establishes is the implemented dependency mechanism and its exact work reduction on this family.

4.4 Library retrieval and induction of useful demands

A production index should first use deterministic structure: conclusion head, constants, type constructor, theorem arity, and normalized argument positions. An inexpensive text ranker can supplement these features. Neural ranking is optional; it is never proof evidence. After retrieving a declaration, instantiate it through Lean’s elaborator, generate its side goals, and attach its actual proof term. Never trust a textual theorem statement supplied by a model.

Useful demands need not be the final goal. Sources include a missing denominator condition, the guard of a conditional rewrite, the precondition of an induction step, and a sign condition requested by the nonlinear solver. Give these consumers a small number of ranked requests rather than internalizing every retrieved theorem globally. Cache failed instantiations by the exact context and budget, but invalidate failures when relevant facts change.

Demand search, like any relevance filter, can miss a surprising intermediate lemma. Reserve a fair lane for ordinary instantiation and gradually widen the retrieval and term-generation bounds. The practical objective is to find a short proof cheaply, not to turn a relevance heuristic into an unjustified completeness claim.

5 PolyCone: search for nonlinear inequality certificates

5.1 Certificate language and its soundness

Let $p, g_1, \dots, g_m, f_1, \dots, f_r \in \mathbb{Q}[x_1, \dots, x_n]$. Assume the current context proves $g_i(\vec{x}) \geq 0$ and $f_j(\vec{x}) = 0$. A certificate for $p(\vec{x}) \geq 0$ is the exact polynomial identity

$$p = \sum_{k=1}^s \lambda_k q_k^2 \prod_{i=1}^m g_i^{e_{ki}} + \sum_{j=1}^r h_j f_j, \quad \lambda_k \in \mathbb{Q}_{\geq 0}, \quad e_{ki} \in \mathbb{N}, \quad (1)$$

where q_k, h_j are rational polynomials. Repetition of a factor is permitted; the empty product is 1. The equality multipliers h_j may have either sign.

Proposition 5.1 (Cone-certificate soundness). *If identity (1) holds and all of its sign and equality hypotheses hold at a real valuation, then p is nonnegative at that valuation.*

Proof. Every square is nonnegative. Products of the nonnegative g_i , the square, and λ_k are nonnegative, so their sum is nonnegative. Each $h_j f_j$ vanishes at the valuation. Evaluating the polynomial identity gives the claim. $\square$

This proof is small compared with the search for the identity. The implemented checker independently reconstructs both sums using rational sparse-polynomial arithmetic, checks factor indices and coefficient signs, and compares the result with p exactly. It does not consult the LP solver or a numerical residual threshold. Its input hypotheses remain hypotheses: a Lean adapter must supply their actual proofs in the current scope.

5.2 Finite dictionary construction

For degree cap D , generate a bounded collection of columns

$$c_k = q_k^2 \prod_i g_i^{e_{ki}}, \quad \deg c_k \leq D.$$

Begin with monomial q_k of degree at most $D/2$. Add binomial candidates suggested by the target: unit sums and differences, exact rational square-root ratios of positive even-degree terms, and completing-square expressions inferred from cross coefficients. If

$$au^2 + cuv + bv^2$$

appears, with $a > 0$, try $q = u + (c/(2a))v$; when $b > 0$, also try the opposite orientation. This is a proposal mechanism, not a rule asserting that the remaining polynomial is nonnegative.

The completing-square proposal matters in practice. The initial heuristic recognized $(x - 2/3)^2$, but a positive extra constant obscured its coefficient ratio in $(x - 2/3)^2 + 1/97$. Adding the cross-coefficient construction recovered the intended square. A regression test now covers this case. The unsuccessful initial search returned *unknown*; it did not accept an approximate proof.

Form products of known nonnegative constraints subject to the same degree cap. Include columns mf_j for monomials m of degree at most $D - \deg f_j$; their multipliers are unrestricted rational variables. Deduplicate identical columns and retain their construction recipes. Avoid expanding the global Cartesian product when most constraints concern unrelated variables: partition by variable support and request cross-component products only when the target connects those components.

The shipped implementation uses a simple bounded enumeration and a limit of 4,000 generators. It is a research implementation, not an optimized sparse SOS package. A production implementation needs a separate bound on enumeration work, monomial count, coefficient bit length, and external-solver time; a cap on successfully admitted columns alone is insufficient as a denial-of-service or worst-case complexity control.

5.3 Search becomes linear algebra with an untrusted oracle

Expand the columns and target in a common monomial basis. The certificate search is

$$A\lambda + Bh = b, \quad \lambda \geq 0, \quad (2)$$

where the h coordinates are free. The Python prototype calls SciPy's `linprog(method="highs")` to propose a solution [11]. It minimizes the sum of cone coefficients as a simple heuristic. A production objective should also penalize large polynomial terms, expensive guard proofs, and predicted replay cost; minimizing coefficient sum is not proof-size minimization.

Floating-point coefficients are converted to rational candidates with bounded denominators and checked exactly. If that fails, solve the proposed active support with exact SymPy linear algebra, set any remaining free parameters to zero, and check the result again. A negative reconstructed coefficient, an inconsistent support, or a failed identity causes rejection. No tolerance is used at the acceptance boundary.

```
polycone(target, inequalities, equalities, D):
    columns := degree-bounded products and proposed squares
    add ideal columns with unrestricted coefficients
    numerical_candidate := solve_linear_program(columns, target)
    if no candidate: return unknown
    rational_candidate := reconstruct(numerical_candidate)
    if exact_check(rational_candidate): return certificate
    support_candidate := exact_solve(numerical_support)
    if exact_check(support_candidate): return certificate
    return unknown
```

The choice of zero values for free reconstruction parameters is incomplete: a different rational point on the same support could have nonnegative coefficients. Similarly, numerical infeasibility does not establish nonnegativity failure and may not even establish infeasibility of the exact finite problem. These are performance limitations only because every successful result passes an independent exact check.

5.4 Worked certificates

Several tested cases explain the range of the certificate language. For the two-dimensional Cauchy–Schwarz inequality,

$$(a^2 + b^2)(c^2 + d^2) - (ac + bd)^2 = (ad - bc)^2.$$

For variance in three variables,

$$x^2 + y^2 + z^2 - xy - yz - zx = \frac{1}{2}(x - y)^2 + \frac{1}{2}(y - z)^2 + \frac{1}{2}(z - x)^2.$$

Both follow from squares without order assumptions on the variables. For $x, y \in [0, 1]$,

$$xy(1 - x)(1 - y) \geq 0$$

uses four known nonnegative factors. The identity has degree 4 and is rejected by the prototype’s degree-2 search before LP construction.

Equality hypotheses contribute differently. Under $y = x$,

$$y^2 - 2x + 1 = (x - 1)^2 + (x + y)(y - x).$$

The second term need not be nonnegative; it vanishes because of the equality. Treating all certificate multipliers as nonnegative would unnecessarily lose this useful algebraic flexibility.

5.5 Theory cooperation through generated inequalities

The plugin should return useful consequences before it can prove the final goal. Suppose $l_x \leq x \leq u_x$, $l_y \leq y \leq u_y$, and a shared atom z is certified equal to xy . The four products of interval distances yield

$$\begin{aligned} z &\geq l_x y + l_y x - l_x l_y, & z &\geq u_x y + u_y x - u_x u_y, \\ z &\leq u_x y + l_y x - u_x l_y, & z &\leq l_x y + u_y x - l_x u_y. \end{aligned}$$

For instance, the first follows from $(x - l_x)(y - l_y) \geq 0$. These inequalities are linear in the shared atoms x, y, z and can strengthen an existing linear arithmetic solver. Their derivations remain polynomial certificates with explicit bound dependencies. The proposal is to generate such cuts on demand, not to replace the linear engine.

For natural numbers and integers, use their actual semantics. Natural subtraction is truncated; machine arithmetic may wrap; division and remainder have domain-specific rules. Reify into an ordered field only after proving the necessary cast and operation correspondence. A real-arithmetic relaxation can supply a sufficient certificate for an integer theorem after a checked embedding, but it cannot justify an integer witness or an integer-specific cut without an additional argument.

Strict inequalities require separate bookkeeping. A positive rational margin in a certificate is enough to prove strict positivity; nonnegativity alone is not. Rational-expression normalization requires denominator conditions. These conditions are obligations on the fact graph, not informal side notes in an external certificate.

5.6 Limits and a concrete SOS escalation

The number of monomials of total degree at most D in n variables is $\binom{n+D}{D}$. Even before proof reconstruction, indiscriminate degree and variable growth can make the coefficient problem too large. Apply cheap symbolic factorization and variable separation first, then expand a bounded basis in increments, recording which expansion helped.

A finite binomial-square dictionary is strictly more restrictive than arbitrary sum-of-squares search. An optional escalation can introduce a Gram matrix $Q \succeq 0$ with $p = v^\top Q v$ or its constrained analogue, solve for Q using a semidefinite optimizer, and reconstruct an *exact* rational factorization

$$Q = L \operatorname{diag}(d_1, \dots, d_t) L^\top, \quad d_i \geq 0.$$

The resulting weighted squares fit the same certificate checker. A numerical claim of positive semidefiniteness is not accepted; an exact factorization and identity are required. Singular matrices and rational reconstruction can make this escalation fail. No SDP solver is implemented in this package.

The tested Motzkin polynomial

$$M(x, y) = x^4 y^2 + x^2 y^4 + 1 - 3x^2 y^2$$

is nonnegative by arithmetic–geometric mean applied to the three nonnegative terms $x^4 y^2$, $x^2 y^4$, and 1. The finite unconstrained square dictionary does not certify it. Its status is therefore **unknown**, despite truth. No bounded cone in this report is advertised as a complete real-algebraic decision procedure. Rational multipliers, richer positivity theorems, or other proof methods require explicit new certificate forms and soundness lemmas rather than a more generous floating-point tolerance.

6 Bernstein certificates on rational boxes

6.1 A complementary proof language

A local box constraint can make a positivity proof easier than a global square decomposition. Map $x \in [a, b]$, $a < b$, to $t = (x - a)/(b - a) \in [0, 1]$. For degree d write the polynomial exactly as

$$p(a + (b - a)t) = \sum_{k=0}^d b_k B_{k,d}(t), \quad B_{k,d}(t) = \binom{d}{k} t^k (1 - t)^{d-k}.$$

If every $b_k \geq 0$, each summand is nonnegative on the interval. Products of these bases give a certificate for a multivariate box.

For power coefficients a_j on $[0, 1]$, the conversion formula is

$$b_k = \sum_{j=0}^k a_j \frac{\binom{k}{j}}{\binom{d}{j}}.$$

To verify it, use $t^j = \sum_{k=j}^d \binom{k}{j} \binom{d}{k}^{-1} B_{k,d}(t)$, which follows by expanding the binomial theorem for the remaining degree $d - j$. The multivariate conversion is its tensor product.

The prototype discovery routine computes the coefficients after an affine change of variables. Its checker takes a deliberately different route: it reconstructs every basis term directly in the

original coordinates,

$$\binom{d}{k} \frac{(x-a)^k (b-x)^{d-k}}{(b-a)^d},$$

and checks the resulting polynomial identity exactly. Both routines share the sparse rational arithmetic layer, so their implementation independence is not complete. Differential tests against SymPy address that shared layer but do not constitute formal verification.

6.2 Subdivision is a proof tree, not sampling

If some coefficients are negative, bisect a box coordinate and try both children. A successful certificate is a complete binary tree of boxes, with nonnegative-coefficient leaves. The checker verifies the root box, each cut's location, the exact two child boxes, and every leaf identity. A gap in the cover invalidates the entire certificate even if every retained leaf looks correct. Testing finitely many points is never an acceptance rule.

For

$$p(x) = x^2 - x + \frac{1}{3} \quad \text{on } [0, 1],$$

the degree-2 coefficients are $(1/3, -1/6, 1/3)$, so the initial box does not certify positivity. On $[0, 1/2]$ they become $(1/3, 1/12, 1/12)$, and on $[1/2, 1]$ they become $(1/12, 1/12, 1/3)$. Two leaves prove the entire interval. This example is included in the tests and measurements.

6.3 Why true statements can remain unresolved

The polynomial $(x - 1/3)^2$ is nonnegative, yet midpoint subdivision never makes $1/3$ an endpoint of an interval with dyadic endpoints. On the leaf containing that point in its interior, all Bernstein basis functions are strictly positive there. If every coefficient were nonnegative and their sum were zero at that interior point, every coefficient would have to vanish. The polynomial would then vanish identically on the leaf, which it does not. Thus this particular subdivision strategy cannot finish, not merely because the tested depth cap is too small.

This is a useful division of labor: the cone routine can recognize the square, while a naive box routine cannot. Conversely, box constraints can certify polynomials that a small global dictionary misses. Route a problem according to observed structure, preserve both alternatives, and report the actual certificate family that succeeded.

A production splitter can use rational root isolation or symbolic factorization to expose zero sets before subdivision. That is a concrete improvement, but it requires certified root information and complete domain decomposition. Adaptive numerical guesses about root locations remain proposals until those obligations are discharged.

7 Induction as a search action

7.1 Generate a justified induction problem

More ground reasoning is not a substitute for a structural argument. Forge should inspect recursive definitions occurring in the goal, their equation theorems, and the arguments on which recursive calls decrease. This yields a small, ranked set of induction candidates. Structural induction on a list or natural number is the first case; a function-specific induction theorem or a well-founded recursor is used when the recursion requires it.

The algorithm must preserve the distinction between discovering an induction hypothesis and assuming an arbitrary stronger assertion. A generalization creates a *new theorem to prove*, with its own base and step obligations. Only the checked induction principle closes that theorem. An open induction branch cannot contribute its conjectured invariant as a global fact.

A concrete procedure is:

1. Read the target and selected hypotheses as a dependency graph. Identify recursive calls and the local variables in their changing argument positions.
2. Rank induction variables using occurrence in decreasing arguments, recursor size, and expected branch count. Start with one variable, then bounded pairs or a function-specific induction principle.
3. Form a generalization set from variables changed by the recursive calls. Take its dependent-type closure, so reverting a variable does not strand local declarations whose types depend on it.
4. Revert the selected declarations, apply the actual recursor or induction theorem, introduce branch parameters and induction hypotheses, and unfold only relevant equation lemmas.
5. Solve the resulting AND-branches cooperatively. On failure, retain the residual goal to suggest a wider generalization or a missing auxiliary lemma, but do not reuse the unproved assertion as evidence.

A “safe” transformation is not necessarily cheap. Even semantically appropriate induction can create an enormous proof search. Put a hard bound on branch count and total candidate recursors before invoking it.

7.2 Worked accumulator example

Define, mathematically,

$$\begin{aligned} \text{revAcc}([], a) &= a, \\ \text{revAcc}(x :: s, a) &= \text{revAcc}(s, x :: a). \end{aligned}$$

The natural target is $\text{revAcc}(s, []) = \text{reverse}(s)$. A weak induction hypothesis talks only about the accumulator $[]$, whereas the recursive call uses $x :: a$. It does not match.

The changed-argument analysis identifies the accumulator, suggesting

$$\forall a, \quad \text{revAcc}(s, a) = \text{reverse}(s) ++ a. \quad (3)$$

The empty-list case follows from the equations. In the cons case, the induction hypothesis instantiated at $x :: a$ gives

$$\text{revAcc}(x :: s, a) = \text{reverse}(s) ++ (x :: a).$$

The recursive equation for reverse and associativity of append identify this with $\text{reverse}(x :: s) ++ a$. Specializing the now-proved generalized theorem at $a = []$ gives the original target. The complete proof idea is the quantified accumulator invariant, not an unexplained instruction to “try induction harder.”

The library may already contain a theorem that makes this example trivial. That does not invalidate the generalization mechanism, but it changes the benchmark. Tests of structural synthesis must use fresh definitions or control which equivalent high-level lemmas are available. Never count retrieval of an already-proved target as evidence that induction synthesis worked.

7.3 Failure-guided auxiliary lemmas

After an unsuccessful step, compare the recursive call with the induction hypothesis. Mismatched accumulator arguments suggest generalization; mismatched compositions suggest a fusion lemma; polynomial differences suggest an invariant template. A candidate lemma is ranked by term size, number of new quantifiers, and whether its recursive calls have a clear decreasing argument. Try at most a bounded number at each size.

For a function f with an accumulator update $a \mapsto T(a, x)$, templates can include

$$f(s, a) = H(s, a), \quad \mu(f(s, a)) = \mu(a) + J(s), \quad R(f(s, a), s, a).$$

The template vocabulary is built from the goal, equation theorems, and a small approved operator set. The resulting lemma is an OR-alternative with a separate AND proof tree. If its proof closes, inject its specialization into the parent. Otherwise discard it or widen the budget. This avoids mutually recursive “helper lemmas” becoming a circular proof.

8 Polynomial invariant synthesis with exact validation

8.1 From recurrence data to a theorem candidate

For a state transition T and starting state s_0 , an equality invariant $I(s) = 0$ is justified by

$$I(s_0) = 0, \quad I(T(s)) = I(s).$$

The second identity is a convenient sufficient condition. More generally it would suffice to prove $I(s) = 0 \Rightarrow I(T(s)) = 0$, but that is not the certificate language implemented here.

The prototype considers additive polynomial recurrences

$$n' = n + 1, \quad s' = s + p(n), \quad (n_0, s_0) = (0, c), \quad (4)$$

where $p \in \mathbb{Q}[n]$. It proposes a polynomial $Q(n)$ and sets $I(n, s) = s - Q(n)$. The certificate obligations become

$$Q(0) = c, \quad Q(n+1) - Q(n) = p(n). \quad (5)$$

Given a degree bound d , sampling the recurrence at $d+1$ points and interpolating suggests Q . *Sampling is only discovery*. The validator substitutes the polynomial transition into I and compares exact coefficient maps for the universal identity.

A production synthesizer should also support direct coefficient solving. Write $Q(n) = \sum_{i=0}^d c_i n^i$, expand (5), and solve the resulting rational linear system. This removes numerical concerns and avoids a dependence on the availability of inexpensive evaluation samples. The interpolation implementation was chosen here because it makes the separation between plausible evidence and universal checking particularly explicit.

8.2 Example: sum of squares

For $p(n) = (n+1)^2$ and $c = 0$, the discovered expression is

$$Q(n) = \frac{n(n+1)(2n+1)}{6}.$$

Equivalently take the denominator-cleared invariant

$$J(n, s) = 6s - n(n + 1)(2n + 1).$$

Its step difference is

$$\begin{aligned} J(n + 1, s + (n + 1)^2) - J(n, s) &= 6(n + 1)^2 \\ &\quad - (n + 1)(n + 2)(2n + 3) + n(n + 1)(2n + 1) \\ &= 0. \end{aligned}$$

Its base value is zero. Ordinary induction on the number of iterations therefore proves the equality for every reachable state. No extrapolation from finite data is involved after the two identities have been checked.

The tests include a hostile interpolation example: an added high-degree term vanishes at all sampled points but violates the recurrence identity elsewhere. The exact validator rejects the proposed invariant. This is an essential test for any counterexample-guided or sample-based synthesis component.

Proposition 8.1 (Invariant certificate soundness). *Let $s_{k+1} = T(s_k)$. If $I(s_0) = 0$ and $I \circ T = I$, then $I(s_k) = 0$ for every $k \in \mathbb{N}$.*

Proof. The claim for $k = 0$ is the base condition. If $I(s_k) = 0$, then $I(s_{k+1}) = I(T(s_k)) = I(s_k) = 0$. Apply induction. $\square$

8.3 Generalizing the invariant engine

For a polynomial state vector $\vec{x}$, use the template

$$I(\vec{x}) = \sum_{|\alpha| \leq d} c_\alpha \vec{x}^\alpha.$$

Because T is fixed, the coefficients of $I(T(\vec{x})) - I(\vec{x})$ are linear in the unknown c_α . A rational nullspace computation discovers conserved polynomials, with the additional linear constraint $I(s_0) = 0$. Reject the identically zero invariant and rank remaining candidates by relevance to the postcondition. This is an immediately implementable extension, although the shipped discovery routine handles only (4).

A more expressive certificate permits a vector of invariant equalities:

$$I_i(T(\vec{x})) = \sum_j h_{ij}(\vec{x}) I_j(\vec{x}).$$

This proves preservation of their common zero set. Bounded-degree multiplier search reduces the check to exact identities, but jointly discovering the I_j and multipliers is no longer the same easy linear problem. Separate candidate generation from multiplier certification, or fix one side at a time; do not describe the joint bilinear search as linear algebra.

Inequality invariants require base and preservation *inequalities*. The PolyCone plugin can certify these under transition guards and existing invariants. Termination remains a different obligation: an invariant does not prove that a loop or recursive definition terminates. For already accepted Lean definitions, use their existing recursion principles; for a proposed new recursive program, termination must be proved separately.

9 Existential witnesses and counterexample-guided synthesis

9.1 Construct witnesses in the correct scope

For a goal $\exists y : \tau, P(\vec{x}, y)$, the useful output is a term t of *exactly* type τ and a proof of $P(\vec{x}, t)$. Search begins with terms already in scope, constructor applications, affine expressions suggested by equations, and terms built from the relevant theorem signatures. Candidate terms must not depend on variables introduced inside a later branch or a future universal quantifier.

For

$$\forall n \in \mathbb{N}, \quad \exists m \in \mathbb{N}, \quad m > n \wedge m \equiv 0 \pmod{2},$$

a bounded affine grammar can propose $m = 2(n + 1)$. The remaining proof obligations are natural-number arithmetic and divisibility. Construct the existential proof explicitly after those obligations close. A real-valued LP solution for m would not be a natural-number witness; domain checks are part of the construction, not an optional verification pass.

9.2 A concrete CEGIS loop

Counterexample-guided inductive synthesis (CEGIS) alternates candidate proposals and attempted refutations of their universal correctness. For an affine integer grammar $t(\vec{x}) = c_0 + \sum_i c_i x_i$, enumerate bounded integer coefficient vectors or ask an integer solver for coefficients that satisfy a finite sample set. Once a candidate is found, ask the cooperative prover to establish $\forall \vec{x}, P(\vec{x}, t(\vec{x}))$ under the actual assumptions.

```
synthesize(grammar, samples, goal, budget):
  repeat within term-size and coefficient bounds:
    t := candidate fitting every validated sample
    result := prove_universally(goal specialized at t)
    if result is a checked proof:
      return explicit witness t and proof
    if result contains a validated counterexample:
      add it to samples
    otherwise:
      schedule a different candidate or a larger proof budget
  return unknown
```

An external model is not automatically a Lean counterexample. Recheck its values against the encoded operations and assumptions, especially casts, truncating subtraction, division, and finite types. A spurious model should refine the encoding or be ignored; it must not eliminate a valid candidate. Likewise, failure to prove a candidate does not mean it is false.

This module is specified, not implemented in the package. A reasonable first implementation restricts the grammar to constructors, affine arithmetic, and finite tuples, reusing `omega`, `linarith`, and `grind` for residual proofs. The current reference explicitly does not describe `omega` as a complete procedure for every integer-linear problem [12]; the integration should preserve ordinary tactic failure semantics.

9.3 Models guide search; they do not certify theorems

The same discipline applies to learned premise selection, algebraic pattern recognition, and suggestions from a computer algebra system. A proposed factorization becomes useful only after the exact identity is proved. A proposed induction invariant becomes useful only after base and preservation are proved. A proposed witness becomes useful only after its universally scoped specification is proved. This uniform boundary makes the architecture compatible with more ambitious search methods without enlarging the logical trust base merely because a search component is complicated.

10 Certified solver islands and theory combination

10.1 Export a fragment, not an untyped approximation of Lean

A *solver island* is a selected subproblem whose translation and proof reconstruction are supported end to end. Boolean and fixed-width bit-vector islands can use the existing SAT-oriented facilities. A suitable first-order island can use Lean-SMT and its reconstruction path; a clause-based island can use Duper [1, 7, 6]. The island may prove a lemma over shared interface terms rather than the complete original goal.

Each export has a translation ledger: Lean expressions, their solver symbols, their types and operations, and proof obligations connecting the encoding to the original proposition. Universe instantiation, monomorphization, typeclass resolution, and auxiliary assertions must have justified transformations. The Lean-SMT work illustrates why that preprocessing is a substantial part of an integration rather than mere serialization [7].

For an island under assumptions $A_1, \dots, A_k$, reconstruct a proof of $A_1 \rightarrow \dots \rightarrow A_k \rightarrow L$. Apply it to the appropriate scoped proofs to insert L into the main graph. An external `unsat` answer without a supported proof is `unknown`. An external `sat` answer is a model proposal, not necessarily a counterexample to the Lean goal. Unsupported proof rules and unsupported theories must be detected and reported, not silently trusted.

10.2 Do not import unsound combination assumptions

A generic textbook theory-combination procedure is not automatically valid for every Lean type. Finite domains, dependent types, casts, and overlapping arithmetic operators complicate disjointness and cardinality assumptions. Forge avoids relying on a blanket combination theorem: plugins exchange *proved interface facts*, and every final step is checked in Lean. This is sound even when incomplete.

When a plugin wants $u = v$ to reconcile a model, it may request a case split or ask another plugin to prove it. It cannot simply merge those terms. Similarly, an arithmetic atom that abbreviates xy must carry a definition or equality proof; treating it as an unrelated real variable gives only a relaxation, not a bidirectional equivalence of problems.

10.3 Two trust profiles

The default target is **kernel-checked replay**: ordinary proof terms or a proved certificate-checker theorem whose acceptance computation is justified without trusting an external search

program. The intended trusted assumptions are Lean’s ordinary accepted foundations and explicitly audited library axioms. Replay should not introduce `sorryAx`, a newly assumed theorem, or a hidden oracle axiom.

A separate, explicitly named **native-checking profile** may run a certificate checker using native compilation for performance. This has a different trust story. The reference warns that `native_decide` adds the Lean compiler to the trusted part, whereas `decide_cbv` does not require trust in the code generator [12]. Do not summarize both profiles as “only the kernel is trusted.” A SAT certificate and a theorem about its checker are not, by themselves, a complete description of how the checker was actually evaluated.

The precise existing implementation of a delegated tactic must be audited under the pinned version. In strict mode, disable a backend when its available replay path requires the native profile. Certificate size, kernel-replay time, and axiom dependencies belong in the result record alongside search time.

11 Concrete Lean integration plan

11.1 Use the current extension interface

The inspected API has `Lean.Meta.Grind.SolverExtension` and `registerSolverExtension`. The latter registers an extension during initialization and allocates its state. Its methods include internalization, new equality and disequality notifications, an action, consistency-related checks, and a model-combination hook. State access, term marking, fact insertion, and proof explanations are also exposed [2].

The API skeleton below records actual conceptual hook names; the handler bodies and Forge state are **design pseudocode**, not a compiled module.

```
initialize polyExt := registerSolverExtension makePolyState
initialize polyExt.setMethods
  (internalize := collectSupportedArithmetic)
  (newEq       := updateCanonicalAtoms)
  (newDiseq    := wakeNonzeroGuards)
  (action      := boundedCertificateAction)
  (checkInv    := checkInternalBookkeeping)
```

Make the adapter a narrow, version-specific module. It reifies supported expressions with a valuation map, requests a certificate, reconstructs a proof, and submits a proved fact through the appropriate insertion service. The check of internal data-structure invariants is not the mathematical soundness proof of a certificate. Name and test those two responsibilities separately.

Structural induction is better implemented above the local saturation loop: it changes the goal shape and creates dependent subgoals. The resulting branch contexts can each host a `grind` session and its plugin state. This two-level architecture keeps the existing theory engine local while the outer graph tracks recursor applications and generalized lemmas.

11.2 Module boundaries and required contracts

Proposed module	Responsibility	Acceptance contract
Forge.Context	Scoped expression, fact, and obligation graph	No unproved facts; valid dependency scopes
Forge.GrindAdapter	Version-specific extension hooks	Restorable state; exact expression/proof transport
Forge.Demand	Retrieval, instantiation, tabling	Every application elaborates and has acyclic proof parents
Forge.Poly.Reify	Typed polynomial representation	Denotation equality for each reified term
Forge.Poly.Cert	Cone and box certificate checking	Proved checker soundness or explicit proof reconstruction
Forge.Induct	Recursor selection and generalization	Goal refinement through an actual induction theorem
Forge.Synth	Invariants and existential candidates	Base/step or witness specification discharged
Forge.Islands	SAT, SMT, and superposition adapters	Supported translation and proof rules; no raw oracle acceptance
Forge.Replay	Proof minimization and audit	Exact original goal, no remaining metavariables, approved axioms

Reification deserves particular attention. To use a rational polynomial certificate, store a map ρ from polynomial variables to Lean expressions and a proof that the original term equals the polynomial's denotation at ρ . For an ordered-field inequality, every coefficient cast and operation must agree with that denotation. An algebraically correct certificate for the wrong denotation is not a proof of the original statement.

11.3 Libraries and algorithms with specific roles

The initial Lean dependency should be **core Lean plus mathlib**, with Aesop used through the version selected by the pinned dependency graph when available. Reuse `ring` or `ring_nf` for polynomial identities, `norm_num` for rational constants, `positivity` for cheap sign facts, and `linarith`/`nlinarith` for arithmetic leaves and selected certificate reconstruction [13, 14, 5]. Do not invoke all of them blindly on every node.

The research implementation uses Python's `fractions.Fraction` and a small sparse-polynomial representation for checking, SciPy/HiGHS for LP proposals, and SymPy for exact support solving and interpolation. Exact tested versions are recorded with the results. NumPy constructs numerical matrices; its floating-point values never decide certificate acceptance.

Optional integrations are Duper for superposition and Lean-SMT with its supported external solver for SMT proofs. Keep them out of the minimal build until compatibility tests pass. A numerical semidefinite solver can later supply Gram-matrix proposals, but an exact replay path is a prerequisite. An existing e-graph package is not required: `grind` already maintains congruence machinery. In particular, an archived experimental Lean equality saturation project should not be made a critical dependency simply because its name sounds aligned with the design.

11.4 Resource allocation and proof-cost awareness

After the protected baseline attempt, a practical starting policy allocates local bounded actions rather than unbounded calls: demand expansion, a small cone, a shallow induction candidate, a small witness grammar, and an eligible solver island. Rank actions by observed recent payoff, target relevance, cost estimate, and age. Reserve a periodic fair slot for postponed applicable actions. The precise weights are tunable engineering parameters, not probabilistically calibrated guarantees.

For an implementable non-learned initial scheduler, use an eight-slot round with three demand slots, two cone slots, one structural slot, one witness slot, and one eligible external-island slot. Empty slots are donated to the oldest applicable action. Within a slot, rank target-linked obligations before generic ones, then prefer lower estimated cost; use a deterministic identifier to break ties. Every fourth round give the oldest postponed action priority. This is a proposed starting policy, not a measured optimum or a finite-budget guarantee.

Initial per-action limits can be explicit: expand at most 32 theorem instances per demand quantum; try cone degrees 2, 4, and 6 with at most 4,000 columns; try at most two induction candidates per structural quantum and induction depth at most two; enumerate at most 256 witness proposals per quantum with term size at most six. Track external jobs separately under wall-clock and output-size limits. Every quantum also consumes the common parent budget, so nested actions cannot multiply the allowed work by repeatedly resetting their local limit.

Keep a monotone context epoch alongside each immutable exported problem. An asynchronous result is reconstructed only after checking that its expression mapping and hypothesis dependencies still apply. Never reuse an old mutable metavariable assignment merely because its printed goal matches the current one. If transport to the new context cannot be justified, discard the result or rerun the action. These rules are especially important when induction, backtracking, and external jobs overlap.

Count the cost of *reconstruction*. An apparently fast oracle that emits an enormous proof can lose to a slower search with a small certificate. Use support minimization, common subexpression sharing, and proof-DAG trimming. Keep unsat-core or used-premise information as a proposal for minimization, then replay the reduced proof to verify that no necessary dependency was removed. Hash-consing expressions reduces duplication but does not excuse missing scoping checks.

All external requests require limits on time, memory, serialized input/output size, integer bit length, and parser depth. Do not evaluate returned text as code. A well-formed but gigantic certificate is a resource risk even when its mathematics is harmless. Permit cancellation between checker steps and return an unresolved result on resource exhaustion.

11.5 User-facing syntax and explanations

The following is **proposed syntax**, not syntax accepted by the delivered package:

```
by
  forge (mode := conservative)
    (inductionDepth := 2)
    (polyDegree := 4)
    (witnessSize := 6)
    (external := false)
```

A companion *forge?* should suggest a stable replay script or an explicit certificate invocation. Users should see which premises were used, which induction variable was selected, which parameters were generalized, and what witness was constructed. Failure diagnostics should distinguish a missing sign guard, a degree cap, unsupported reification, an unproved generalized lemma, an unavailable backend, and an exhausted search budget.

The report's generated Lean files are a narrower first artifact: explicit proofs for saved certificates. They contain no search implementation of *forge*. Hiding that distinction behind an umbrella import would misrepresent the state of the project.

12 Soundness obligations for the whole system

Proposition 12.1 (Compositional soundness target). *Suppose every fact node contains a Lean proof in its recorded context; every refinement node is justified by an accepted Lean theorem or recursor; every certificate adapter proves its exact translated conclusion; and every scope transfer is checked. Then a closed root of the acyclic proof graph supplies a proof of the original goal, independently of how search candidates were selected.*

Proof. Proceed in topological order through the proof graph. Base facts are valid by assumption. At a theorem-application node, apply its proof to the previously constructed premise proofs. At a refinement node, combine the checked branch proofs with its theorem or recursor. At a certificate node, use the adapter's proved conclusion. Scope transfer abstracts and instantiates only validated dependencies. The final node therefore inhabits the original proposition. $\square$

This is a design-level argument, not a mechanized proof of the present Python implementation. Lean integration must instantiate each hypothesis of the argument. An unsound term translator is not repaired by an exact polynomial checker. A sound checker is not repaired by a replay path that assumes its result through an oracle axiom. An acyclic certificate DAG is not sufficient if a proof parent was imported from the wrong branch.

Testing should deliberately attack those boundaries. Mutate a denominator condition; swap two polynomial variables; replace a strict inequality with a weak one; change an exponent; reverse an equality transport; remove a branch from a box cover; alter a typeclass instance; point a proof node to a future node; and replay under a different local context. The intended result is rejection, not a best-effort reinterpretation of the malformed evidence.

13 Experiments actually performed

13.1 Environment and reproducibility

The execution environment was Linux x86_64 with Python 3.13.5, NumPy 2.3.5, SciPy 1.17.0, SymPy 1.14.0, and pytest 9.0.2. The synthetic benchmark uses seed 20260914. Numerical-library thread environment variables were set to one for the recorded run. Full version strings are in `results/benchmarks.json`. The test run completed with **73 passed in 1.11 seconds**.

No `lean`, `lake`, or successful Lean execution route was available. The supplied Lean comparison runner therefore recorded `not_run`, with that reason, rather than manufacturing zero failures or baseline timings. Source inspection, Python execution, and generated Lean source are three different forms of evidence and remain separately labeled throughout the package.

The benchmarks were executed once for diagnostic purposes; there are no repeated-trial confidence intervals or hardware-normalized performance claims. The exact combinatorial counts and algebraic acceptance results are more informative here than submillisecond timings.

13.2 Summary of observed outcomes

Experiment	Outcome	Interpretation
Pytest suite	73 passed	Includes parametrized, randomized, negative, and mutation tests
Named nonlinear cases	9 accepted / 12	Two true cases unresolved; one deliberately false case unresolved
Generated product-cone cases	100 / 100	Constructed inside the covered cone; not representative problem sampling
Separate false-polynomial controls	0 accepted / 24	All returned unknown; exact negative witnesses checked separately
Bernstein cases	43 accepted / 45	One true nondyadic-zero case and one false case unresolved
Additive recurrence invariants	49 / 49	Polynomial increments; base and universal step identities checked
Horn growth family	5 depths	Both strategies prove all goals; search work differs sharply
Actual Lean comparison	Not run	No measured superiority over grind

The 109 accepted nonlinear certificates are the nine named successes plus the 100 generated cases. They are not 109 previously unsolved Lean theorems. The 49 invariants concern a deliberately restricted polynomial recurrence class, not arbitrary recursive Lean programs.

13.3 Demand search: counts before speed claims

Depth	Forward facts	Demand atoms	Forward ms	Demand ms
4	31	5	0.416	0.128
8	511	9	6.870	0.232
10	2,047	11	35.032	0.293
12	8,191	13	153.160	0.379
14	32,767	15	846.283	0.478

The number of proof nodes in both returned certificates is $d + 1$ at every depth. Thus the search-state reduction is not achieved by silently omitting necessary proof steps. At depth 14, forward search generated 32,766 rule instances and demand search generated 14; each yielded the same 15-node chain. The toy rule order intentionally makes the forward target last at its depth. A different forward strategy, or the actual **grind** implementation, can behave differently.

Correctness testing extends beyond this favorable family. The tests exercise pure dependency cycles, cycles with a base fact, multiple-premise AND-rules, wrong goals, invalid proof-parent indices, depth/fact limits, and randomized finite ground Horn systems compared across both search implementations. These tests support the implementation's behavior, not a completeness claim for general theorem proving.

13.4 Nonlinear cases, including unresolved truths

The named successful cases are a shifted square, two-variable AM–GM, a cubic box product, a quartic box product, an equality-ideal identity, a shifted square with rational margin, two-dimensional Cauchy–Schwarz, a product of three lower bounds, and three-variable variance. The quartic product under a degree-2 cap and the Motzkin polynomial remain unresolved. A nearly nonnegative but actually false shifted square also remains unresolved, as required.

For the generated set, each target is a positive rational sum of six products of constraints from $x, 1 - x, y, 1 - y$, with product degree at most four. This construction intentionally places the targets inside the chosen certificate cone. It tests discovery, rational reconstruction, and checking on diverse coefficient combinations; it does not estimate performance on randomly chosen mathematical inequalities. The median recorded discovery time was about 21.5 milliseconds and the largest about 103.6 milliseconds for this set. These are single-run local measurements, not a speed comparison with Lean.

Each of the 24 additional negative controls has the form $(x - a)^2 - \varepsilon$ with $\varepsilon > 0$ rational. The benchmark separately checks its exact negative value at a . The cone routine itself returns `unknown`; it does not return a certified refutation. Tests also corrupt weights, remove sign assumptions, change targets by tiny rational quantities, and introduce invalid factor indices. The checker must reject those corruptions even if a numerical optimizer would regard the residual as negligible.

13.5 Bernstein and invariant evidence

The Bernstein set contains the worked two-leaf positive polynomial, a box product, a dyadic zero, a nondyadic zero, a false polynomial, and 40 strictly positive rational quadratics. The 43 accepted cases have complete checked subdivision trees. A test corrupts the box-cover geometry to ensure local positivity does not masquerade as a complete proof.

The invariant set contains nine increments $(n + 1)^k$ for $0 \leq k \leq 8$ and 40 fixed-seed random rational polynomial increments with varied starting values. Every accepted proposal passes exact base and transition checks. The median recorded synthesis time was about 4.1 milliseconds. This includes interpolation and validation but not Lean recursor application, source elaboration, or kernel checking.

The package exports the nine named cone certificates and all 49 invariant certificates as JSON. A separate reload-and-validate script accepted all 58 saved records. The additional generated certificates and box trees are reproducibly rediscovered and checked by the benchmark script; they are not all serialized as separate proof artifacts.

13.6 What the Lean export does and does not establish

The exporter emits nine inequality theorems using explicit nonnegative terms, rational arithmetic, and polynomial identities. It also emits 49 recurrence proofs using a generic induction lemma for iteration and the generated exact base/step expressions. The files are `lean/Forge/Certificates.lean` and `lean/Forge/Invariants.lean`.

They contain no deliberate admissions or new axioms. Nevertheless, the source was not compiled, so it is not a verified Lean deliverable. Elaboration details, tactic API changes, or dependency integration may require corrections. The right acceptance criterion is a successful build followed by an axiom and proof audit, not the plausible appearance of generated source. The included runner leaves this evidence gap explicit.

14 A fair evaluation for a real Forge implementation

14.1 Benchmark strata and leak prevention

A subsequent Lean evaluation should use distinct strata: standard `grind` regressions; nonlinear inequalities; fresh recursive definitions needing induction; accumulator and fusion lemmas; existential constructions; Boolean and bit-vector goals; quantified first-order problems; and mixed goals in which one technique supplies a lemma to another. Keep easy goals, failed goals, and unsupported goals, not only examples chosen after an extension succeeds.

Partition by theorem family and source module rather than individual nearby lemmas. Remove the target theorem from any retrieval index. Check for equivalent lemmas, renamed copies, and specialization leakage. A synthesis benchmark whose target is already available as a library fact primarily evaluates retrieval. That can be useful, but it is a different experiment.

Record the complete imported environment, theorem attributes, simplification sets, seeds, solver versions, and tactic configuration. Reset proof state between runs; do not let a successful trial add a lemma that helps a later baseline run.

14.2 Baselines and ablations

Compare stock `grind`, explicitly configured `grind` variants, relevant individual tactics, and a straightforward portfolio. Then compare Forge with one extension removed at a time: demands, nonlinear cuts, induction generalization, invariant synthesis, witnesses, or external islands. Compare the cooperative version with the same engines run independently. Otherwise, a gain could come entirely from adding a known tactic rather than from the proposed information exchange.

Use two resource protocols. Under *fixed total resources*, all systems receive the same wall-clock limit, memory limit, core count, and appropriately reported heartbeat budget. Under *additive fallback resources*, allow the unchanged baseline budget plus a declared extension budget, reporting both budgets and the extra cost. Neither protocol substitutes for the other.

Metrics should include checked solved goals, timeouts, elaboration errors, unsupported translations, proof bytes, reconstruction time, kernel replay time, and unexpected axioms. Report paired wins and losses, not only aggregate success. Repeat timings in randomized order and report confidence intervals or paired distributions. Include proof checking in the main end-to-end result, while retaining search-only timing for diagnosis.

The supplied comparison script is much smaller: it tries existing tactics on nine exported certificate contexts, records compiler diagnostics, and optionally builds the replay library. It is a smoke-test scaffold, not an implementation of this full protocol.

15 Development sequence and acceptance gates

Stage 1: reliable replay. Compile the generated Lean files, fix any elaboration issues, and audit their axioms. Implement a typed polynomial reifier and explicit certificate replay before connecting an external oracle. Reject malformed certificates and wrong-context assumptions in Lean tests. A successful stage means that a saved certificate proves the exact intended Lean theorem, not that discovery is fast.

Stage 2: nonlinear integration. Connect the bounded LP proposal service to the reifier and checker. Add the goal-sensitive square dictionary, product cuts, and failure diagnostics. Start with real polynomial inequalities and explicit hypotheses; postpone complicated casts and rational expressions until their transport lemmas are tested. Measure against `nlinarith` and stock `grind`, including cases they already solve.

Stage 3: structural coordination. Implement the scoped obligation graph, tabled demands, and one-variable structural induction with changed-argument generalization. Port additive invariant synthesis and prove its recurrence transport. Demonstrate mixed examples in which synthesized structure feeds facts to the algebraic engine. Keep a replay script for every success.

Stage 4: richer synthesis and delegation. Add typed witness grammars, selected solver islands, and optional broader certificate search. Each backend must pass a proof-rule coverage matrix and strict/native trust-profile audit. A missing or incompatible backend should disable only its lane.

Stage 5: validated deployment. Run the controlled repository-scale benchmarks, tune scheduling on a separate training set, preserve stock behavior in conservative mode, and publish wins, regressions, unsupported cases, and proof-checking costs. Do not release an experimental always-on policy merely because it improves an aggregate score on its tuning examples.

These are dependency-ordered milestones, not a promised schedule. The highest risk is the typed, scoped, version-compatible bridge into Lean and the quality of the cooperative search policy, not the small algebraic checker alone.

16 Conclusion

The credible path beyond `grind` is to supply the proof structures and certificates that local saturation lacks, while retaining its mature core. Forge makes that path concrete: demand-directed obligations, exact nonlinear certificates, complete box proofs, justified induction generalization, polynomial invariants, typed witnesses, and narrowly translated solver islands. Its organizing rule is that speculative search may be aggressive but every logical consequence must have a checkable derivation in its actual scope.

The executable work supports four parts of this proposal and exposes meaningful limitations. It does not establish a finished Lean tactic or measured dominance over `grind`. The next decisive evidence is a typed Lean implementation with successful kernel replay and a paired, controlled evaluation. The supplied source, saved certificates, tests, and integration contracts make that a specific engineering program rather than a vague aspiration.

A Certificate formats and independent checking

A.1 Sparse polynomials

A polynomial stores a positive variable count and a sorted list of distinct exponent vectors with nonzero rational coefficients. Variables are positional. Exponent vectors have exactly the declared dimension and contain nonnegative integers. Construction rejects noncanonical encodings instead of silently merging duplicate entries supplied in a purported certificate.

For example, $x^2 - 2xy + y^2$ is represented schematically as

```
{
  "nvars": 2,
  "terms": [
    [[0, 2], "1"],
    [[1, 1], "-2"],
    [[2, 0], "1"]
  ]
}
```

All fractions in the exported JSON are exact strings such as "1/97". The checker does not parse a decimal into a claim of approximate equality. An external-facing production parser should additionally bound sizes before constructing these values.

A.2 Cone and invariant records

A cone record contains the target polynomial, ordered lists of nonnegative and zero hypotheses, and a certificate. Each cone term contains a rational weight, a square polynomial, and a list of factor indices. An index can occur more than once. The equality part contains one arbitrary polynomial multiplier per zero hypothesis. The exact order is part of the certificate's meaning.

An invariant record contains an invariant polynomial, an initial rational state, and one transition polynomial per state coordinate. The reload checker tests both base evaluation and exact polynomial substitution. It is independent of the interpolation and LP discovery functions. It is not independent of the shared rational-polynomial implementation, a limitation addressed by tests rather than concealed as formal verification.

The essential cone-checking logic is the following executable pattern:

```
acc = zero_polynomial
for term in certificate.terms:
    reject unless term.weight is rational and nonnegative
    p = term.weight * term.square**2
    for i in term.factors:
        reject unless i is a valid hypothesis index
        p *= nonnegative_hypotheses[i]
    acc += p
for multiplier, equality in paired_ideal_entries:
    acc += multiplier * equality
accept exactly when acc == target
```

The actual implementation also checks dimensions, array lengths, canonical polynomials, and malformed input. Exceptions representing invalid evidence lead to rejection. Resource-exhaustion behavior is an additional production requirement, not a theorem proved about the prototype.

A.3 Horn and box records

A Horn proof node records its ground conclusion, the rule index or input-fact marker, a complete ground substitution, and earlier premise-node indices. The checker verifies the instantiated rule body and head against those exact nodes. Trimming retains only ancestors of the root, preserving

the topological order. A dependency cycle in the search graph may be legitimate, but the final proof certificate cannot justify itself through that cycle.

A Bernstein certificate is either a leaf with a box, coordinate degrees, and all basis coefficients, or a split with an axis, interior rational cut, and both children. Every basis index must appear exactly once in canonical order. The checker reconstructs the polynomial at each leaf and checks the complete box cover recursively.

B Reproducing the package

From the extracted package root, the Python path is:

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r prototype/requirements.txt
bash scripts/reproduce.sh
```

For the exact versions used in this run, substitute `prototype/requirements-tested.txt`. That file is a tested-version snapshot, not a promise that binary wheels exist for every platform.

Individual actions can be run from `prototype/`:

```
python -m pytest -q
python run_benchmarks.py --output ../results
python validate_artifacts.py --results ../results
python export_lean.py --results ../results --output ../lean
```

The reproduction script regenerates measurements and certificates; wall-clock timings naturally vary. It does not download or install Lean. To attempt the uncompiled replay project on a machine with Lean/Lake and network access:

```
cd lean
lake update
lake exe cache get
lake build
cd ..
python scripts/run_lean_comparisons.py --timeout 30
```

The runner preserves compiler output and distinguishes timeout from nonzero exit. A nonzero compiler exit can be an elaboration or dependency failure, not necessarily a failed mathematical search. The runtime absence recorded in the supplied results is intentional and is not counted as either a solve or a regression.

The article is built with LuaLaTeX. Its main source is `article/forge.tex`, which inputs the numbered chapter files, `horn-table.tex`, and `references.tex`. No font files are bundled. The build helper updates the numerical Horn table from the saved JSON before running the typesetter.

References

- [1] Lean developers. *The Lean Language Reference: The grind tactic*. Current documentation inspected 14 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Lean developers. *Lean.Meta.Tactic.Grind.Types*. Generated API documentation, inspected 14 September 2026. https://leanprover-community.github.io/mathlib4_docs/Lean/Meta/Tactic/Grind/Types.html.
- [3] Lean developers. *src/Lean/Meta/Tactic/Grind/Action.lean*, tag v4.34.0. Inspected source describes the continuation-based control interface, backtracking, and script reconstruction. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Action.lean>.
- [4] Lean community. *Aesop: White-box automation for Lean 4*. Repository documentation, inspected 14 September 2026. <https://github.com/leanprover-community/aesop>.
- [5] Mathlib contributors. *Mathlib.Tactic.Linarith.Frontend*. Implementation overview and configuration documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [6] Lean community. *Duper*. Repository documentation and feature description, inspected 14 September 2026. <https://github.com/leanprover-community/duper>.
- [7] A. Mohamed, T. Mascarenhas, H. Khan, H. Barbosa, A. Reynolds, Y. Qian, C. Tinelli, and C. Barrett. *lean-smt: An SMT tactic for discharging proof goals in Lean*. arXiv:2505.15796, version 1, 2025. <https://arxiv.org/html/2505.15796v1>. Implementation: <https://github.com/ufmg-smite/lean-smt>.
- [8] E. Wang, S. Chess, S. Szeto, and T. Meek. *Learned Interventions Inside Lean 4's grind*. arXiv:2607.22972, version 1, 2026. <https://arxiv.org/html/2607.22972v1>. Cited as a research report; its experiments are not reproduced here.
- [9] Lean developers. *Lean v4.34.0 release*. Published 14 September 2026; release metadata inspected through the GitHub API. <https://github.com/leanprover/lean4/releases/tag/v4.34.0>.
- [10] Mathlib contributors. Repository revision 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa and its `lean-toolchain` file. <https://github.com/leanprover-community/mathlib4/tree/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa>.
- [11] SciPy developers. *scipy.optimize.linprog*. Official API reference. The local experiment used SciPy 1.17.0 with the `highs` method, not necessarily the version currently displayed by the site. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>.
- [12] Lean developers. *The Lean Language Reference: Tactic Reference*. Sections on `native_decide`, `decide_cbv`, and `omega`, inspected 14 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [13] Mathlib contributors. *Mathlib.Tactic.Ring.Basic*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Ring/Basic.html.
- [14] Mathlib contributors. *Mathlib.Tactic.Positivity.Basic*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Positivity/Basic.html.

Provenance note. External sources establish the baseline, available interfaces, related work, and library behavior. The architecture, derivations, Python code, tests, and reported measurements in this package were developed for this task. Research inspiration is distinguished from executed integration; no third-party research paper, repository archive, or font file is redistributed.